

String Handling Functions

strlen()

strlen () : It is used to count the number of characters present in the given string

Syntax: strlen(str);

Ex: c program to find the length of string using strlen() :

```
#include <stdio.h>
#include <string.h>
void main( )
{
    int len;
    char s[20]="welcome to GCET";
    len = strlen(s) ;
    printf ( "string length = %d \n" , len ) ;
}
```

Output : string length is 15

strcpy()

It is used to copy contents of one string into another string.

Syntax: `strcpy(s2,s1);` **s1** is the source string and **s2** is target/destination string.

Ex: c program to copy one string to another string.

```
#include <stdio.h>
#include <string.h>

main()
{
    char s1[10]= "welcome" ;
    char s2[20];
    strcpy(s2, s1);
    printf(" source string is: %s",s1);
    printf(" destination string is %s",s2);
}
```

Output : source string is :**welcome**

destination string is: **welcome**

strncpy ()

It is used to copy number of character from str2 to str1 .

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str1[10]="rama";
    char str2[10]="raheem";
    strncpy(str1,str2,3);
    printf("%s",str1);
    return 0;
}
```

Output: raha

strcmp()

It is used to compare two strings character by character and returns -1 or 0 or 1.

Syntax : strcmp(s1,s2);

- ☐ If the ascii value of character of the first string is less than second string , it returns -ve value .
- ☐ If both strings are equal , it returns 0.
- ☐ If the ascii value of character of the first string is greater than second string, it returns +ve value.

Ex: c program to compare two strings using strcmp () function.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void main()
```

```
{  
    char s1[] = "abc";  
    char s2[] = "bca";  
    int i;  
    i = strcmp(s1, s2);    // comparing strings s1 and s2  
    if(i==0)  
        printf("both strings are equal");  
    else if(i>0)  
        printf("first string is greater than second string");  
    else  
        printf("first string is less than second string");  
}
```

Output : first string is less than second string

strncmp()

strncmp(): Compares first n characters of both string s1 and string s2.

```
#include<stdio.h>
#include<string.h>
int main()
{
char s1[20] = "Books";
char s2[20] = "Books1";
int n=5;
if (strncmp(s1, s2,n) ==0)
    {
printf("string 1 %s and string 2 %s are equal upto %d characters ",s1,s2,n);
    }
else
    {
printf("string 1 and 2 are different");
    }
return 0;
}
```

Output: string 1 Book and string 2 Book are equal upto 5 characters

strcmpi() or stricmp()

strcmpi(): compare two strings, string1 and string2 by ignore case(lower case or upper case).

```
#include<stdio.h>
#include<string.h>
int main()
{
char s1[20] = "BOOKS";
char s2[20] = "books";
int n=5;
if (strcmpi(s1, s2) ==0)
    printf("string 1 %s and string 2 %s are equal upto %d characters ",s1,s2,n);
else
    printf("string 1 and 2 are different");
return 0;
}
```

Output: string 1 BOOKS and string 2 books are equal upto 5 characters

strcat()

strcat() : it is used to concatenate two strings i.e it appends second string at the end of the first string.

Syntax: strcat(s1,s2);

Example: c program to concatenate two strings

```
#include <stdio.h>
#include <string.h>
void main()
{
    char s1[20] = "good ";
    char s2[10] = "morning";
    strcat(s1,s2);      //concatenates s1 and s2 and resultant string is stored in s1.
    printf("concatenated string is %s",s1);
}
```

Output : result is : good morning

strncat()

strncat():it appends number of character from second string (s2) at the end of the first string(s1).

```
#include<stdio.h>
#include<string.h>
int main()
{
char s1[10] = "Hello";
char s2[10] = "World";
strncat(s1,s2,4);
printf("Output string after concatenation: %s", s1);
return 0;
}
```

Output:Output string after concatenation:HelloWorl

strlwr()

strlwr() : It is used to convert uppercase characters into equivalent lower case characters.

Syntax : `strlwr(s1);`

Example : c program to convert uppercase character into lowercase using `strlwr()`

```
#include<stdio.h>
#include<string.h>
void main()
{
    char s1[6] = "HELLO ";
    printf("the lower case string is: %s",strlwr(s1));
    getch();
}
```

Output : The lower case string is: hello

strupr()

strupr() : It is used to convert lowercase characters into equivalent upper case characters.

Syntax : strupr(s1);

Example : c program to convert lowercase character into uppercase using strupr()

```
#include<stdio.h.
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s1[6] = "hello ";
```

```
    printf("the upper case string is %s",strupr(s1));
```

```
    getch();
```

```
}
```

Output : the upper case string is HELLO

strrev()

strrev() : It is used to reverse the given string.

Syntax: strrev(str); where str is the string variable;

Example : c program to reverse the given string using strrev()

```
#include<stdio.h.
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s1[6] = "hello ";
```

```
    printf("the reversed string is %s",strrev(s1));
```

```
    getch();
```

```
}
```

output : the reversed string is olleh

strdup()

strdup():Duplicated value of string2 is assigned to string1

```
#include<stdio.h>
#include<string.h>
int main()
{
char s1[20] = "books";
char *s2;
s2=strdup(s1);
printf("duplicate string is %s",s2);
return 0;
}
```

Output: duplicate string is **books**

strrchr()

strrchr(): `strrchr(string1, 'o')` returns a pointer to the last occurrence of character 'o' in string1

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main ()
```

```
{
```

```
    const char str[] = "hello welcome";
```

```
    const char ch = 'o';
```

```
    char *ret;
```

```
    ret = strrchr(str, ch);
```

```
    printf("String after %c is : %s\n", ch, ret);
```

```
    return(0);
```

```
}
```

Output: String after o is : ome

strchr()

strchr(string1,string2): returns a pointer to the first occurrence of string2 in string1

```
#include <stdio.h>
#include <string.h>
int main () {
    char str[] = "http://www.tutorialspoint.com";
    char ch = 'o';
    char *ret;
    ret = strchr(str, ch);
    printf("String after %c is : %s\n", ch, ret);
    return(0);
}
```

Output: String after o is : orialspoint.com

strset()

strset(string1, 'B') Sets all the characters of string1 to given character 'B'.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
char s1[10] = "Hello";
```

```
    strset(s1,'B');
```

```
    printf("Output string after replaced: %s", s1);
```

```
return 0;
```

```
}
```

Output: Output string after replaced: BBBBB

strnset(string1, 'B', 3)

strnset(string1, 'B', 3) Sets first 3 characters of string1 to given character 'B'.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
char s1[10] = "Hello";
```

```
    strnset(s1,'B',3);
```

```
    printf("Output string after  replaced first 3 characters: %s", s1);
```

```
return 0;
```

```
}
```

Output:Output string after replaced first 3 characters: BBBlo

strstr()

strstr(string, string2) : returns address of first occurrence of the string2 in the string1, if not found returns null.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[] = "Hello World";
```

```
    char sub[] = "World";
```

```
    char *result = strstr(str, sub);
```

```
    if (result != NULL) {
```

```
        printf("Substring found: %s\n", result);
```

```
    } else {
```

```
        printf("Substring not found\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Output: Substring found at index 12

strspn()

This function returns the index of the first character in str1 that doesn't match any character in str2.

```
#include <stdio.h>
#include <string.h>
int main() {
    char str1[] = "Happy Birthday to you";
    char str2[] = "Happy Birthday Joe";
    printf("\n The position of first character in str2 that doesn't match with that in str1 is
%d", strspn( str1, str2));
    return 0;
}
```

Output:

The position of first character in str2 that doesn't match with that in str1 is 15

strcspn()

This function returns the index of the first character in str1 that matches any of the characters in str2.

```
#include <stdio.h>
#include <string.h>
int main() {
    char str1[] = "Happy Birthday to you";
    char str2[] = "Happy Birthday Joe";
    printf("\n the position of first character in str2 that match with that in str1 is
%d", strcspn( str1, str2));
    return 0;
}
```

Output: the position of first character in str2 that match with that in str1 is 0

atoi()

atoi() (ASCII to Integer) is a standard library function used to **convert a string into an int value**. It skips **leading whitespaces**, checks for an optional **+ or - sign**, Converts consecutive **numeric digits** into an integer and stops conversion at the **first non-digit character**.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    char str[] = " -456abc";
    int num = atoi(str);
    printf("Number = %d\n", num);
    return 0;
}
```

Output: -456

atof()

atof() (ASCII to Float) is a standard library function used to **convert a string into a floating-point (double) value**. It skips **leading whitespace**, handles optional **+ or - sign**, converts digits before and after the **decimal point**, supports **scientific notation** (e.g., 1.23e-3) and stops conversion at the **first invalid character**.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    char str[] = " -45.67xyz";

    double num = atof(str);

    printf("Value = %lf\n", num);

    return 0;
}
```

Output: -45.67000

atol()

atol() (ASCII to Long) is a standard library function used to **convert a string into a long int value**.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    char str[] = " -98765abc";

    long int num = atol(str);

    printf("Value = %ld\n", num);

    return 0;
}
```

Output: -98765

strtok()

strtok() is a standard library function used to **split (tokenize) a string into smaller parts (tokens)** based on specified delimiters. Returns a pointer to the **next token** and returns **NULL** when no more tokens are found.

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "C,Java,Python";
    char *token = strtok(str, ",");
    while (token != NULL) {
        printf("%s\n", token);
        token = strtok(NULL, ",");
    }
    return 0;
}
```

Output:

C

Java

Python